Caminos en Grafos

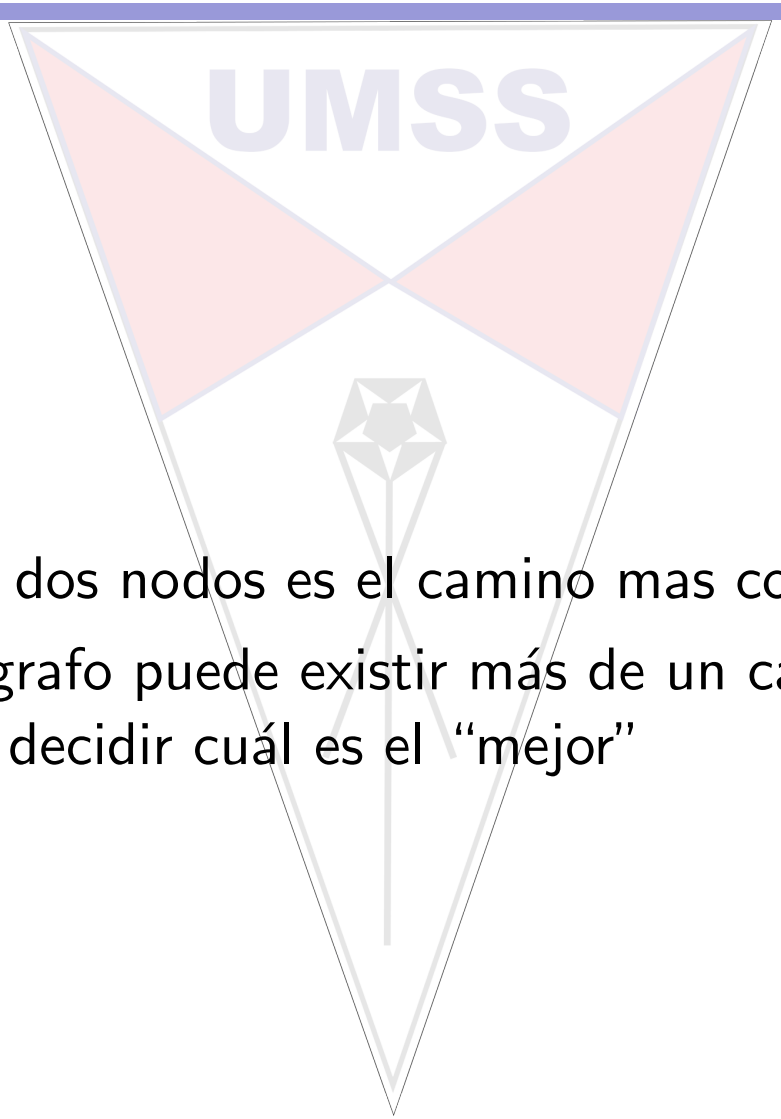
Leticia Blanco

Departamento de Informática - Sistemas
UMSS

13 de mayo de 2025

Contenido

Distancia

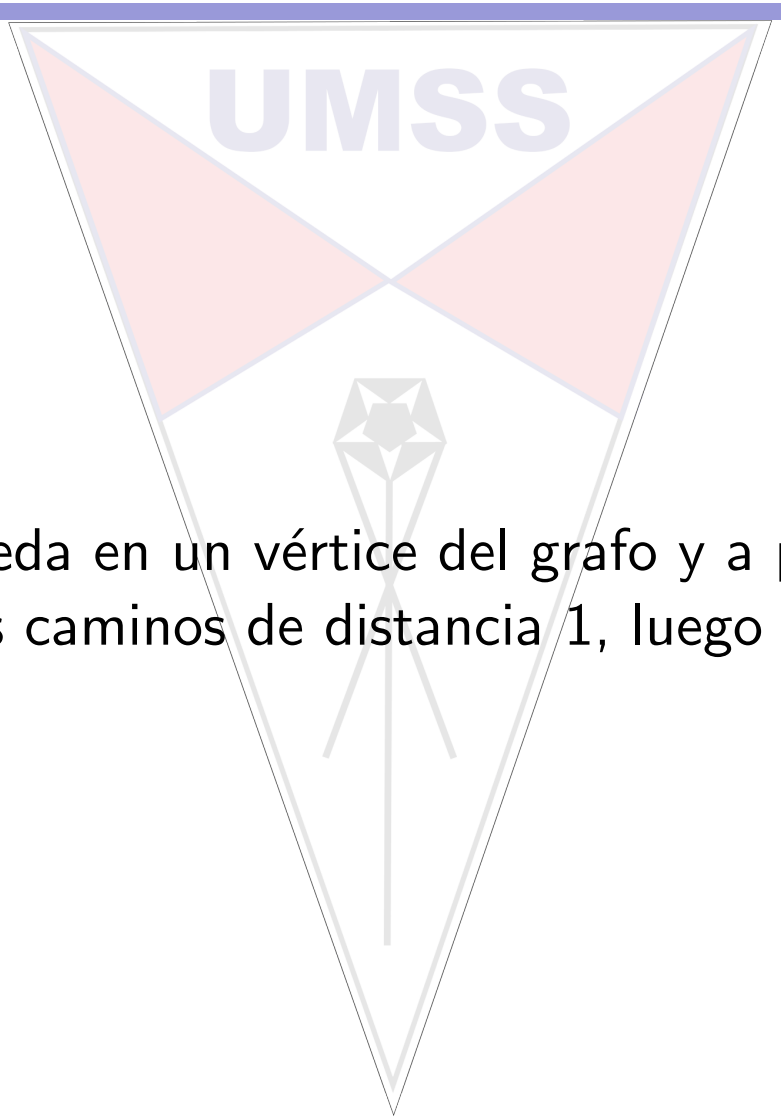


Distancia

La distancia entre dos nodos es el camino mas corto entre ellos

Considerando un grafo puede existir más de un camino entre dos nodos, la tarea es decidir cuál es el “mejor”

BFS

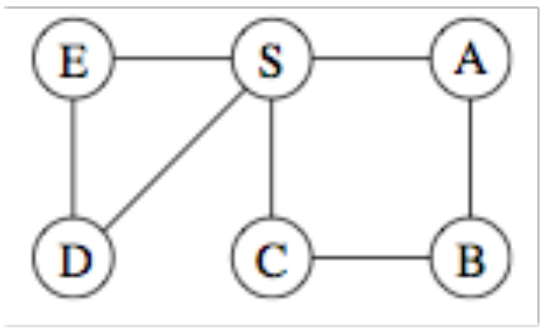


Se inicia la búsqueda en un vértice del grafo y a partir de allí se obtienen todos los caminos de distancia 1, luego de 2 y asi.....

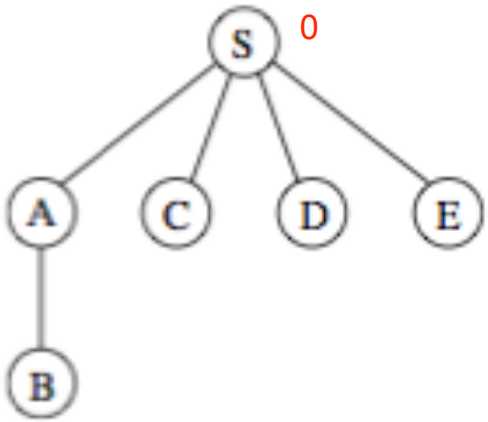
BFS

```
procedure BFS(G,s)
input  : G =(V,E) dirigido o adirigido, s ∈ V
output: ∀ u ∈ V alcanzables desde s, dist(u) es el conjunto de distancias desde s a
        u
for  u ∈ V do
  |   dist(u) = INFINITO
end
dist(s) = 0
Q = [s] //Q es una cola
while  Q != vacia do
  |   u = eject(Q)
  |   for arista (u,v) ∈ E do
  |     |   if  dist(v) == INFINITO then
  |     |     |   inject(Q,v)
  |     |     |   dist(v) = dist(u) + 1
  |     |   end
  |   end
end
```

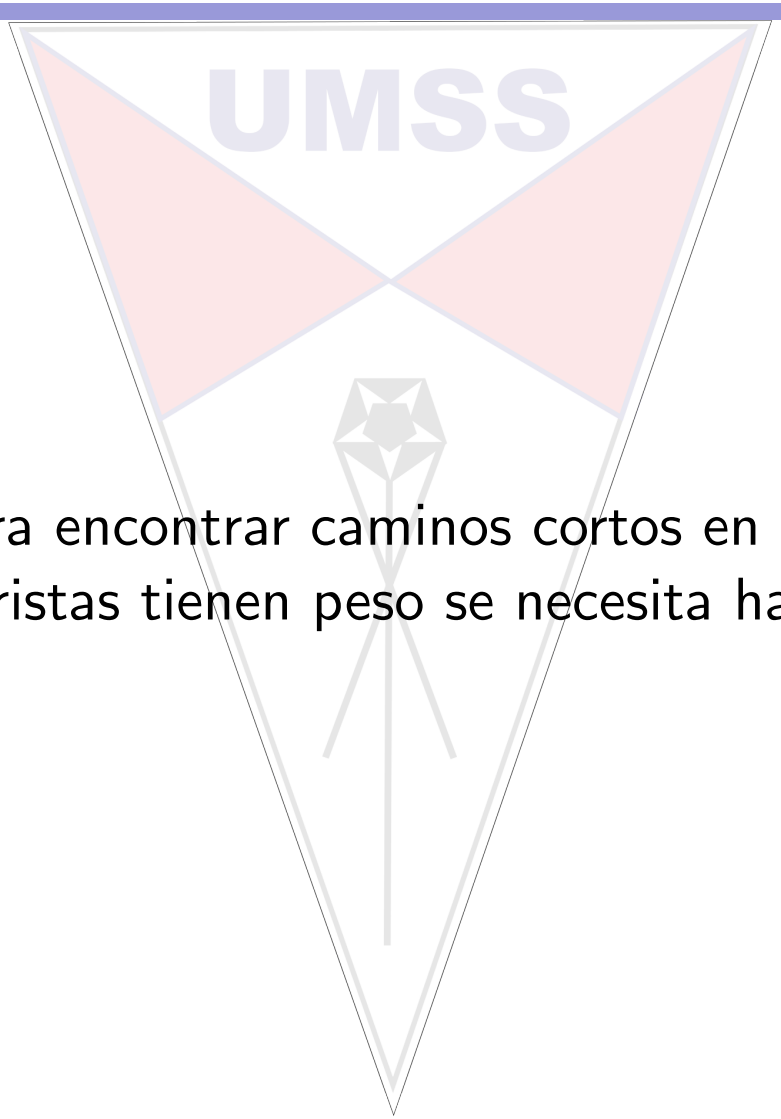
BFS



Order of visitation	Queue contents after processing node
	$[S]$
S	$[A\ C\ D\ E]$
A	$[C\ D\ E\ B]$
C	$[D\ E\ B]$
D	$[E\ B]$
E	$[B]$
B	$[\,]$



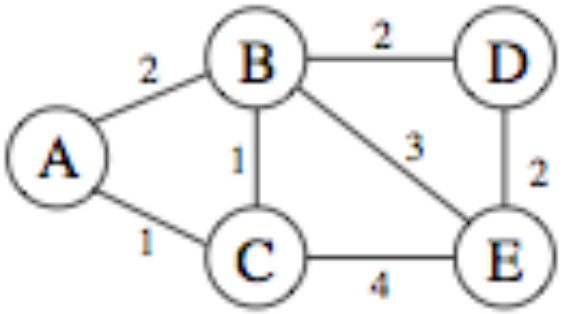
BFS

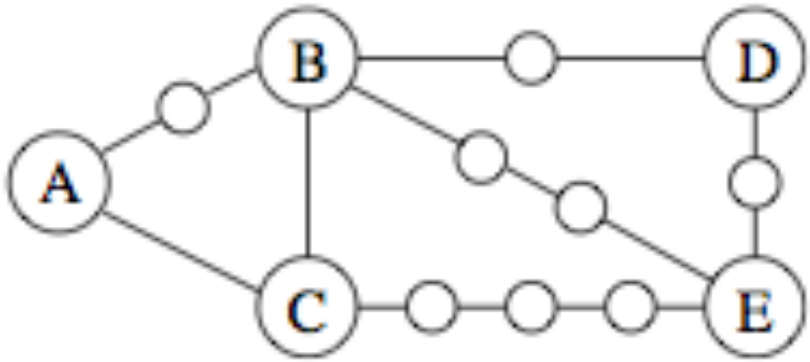


La BFS es útil para encontrar caminos cortos en grafos sin peso, pero cuando las aristas tienen peso se necesita hacer otras consideraciones

Dijkstra

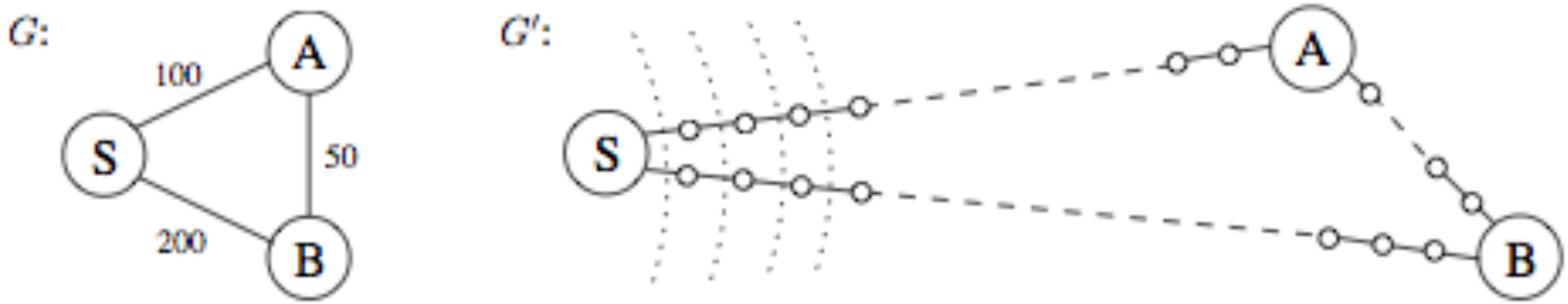
La BFS fue la base para plantear la solucion. Parece natural partir las aristas en muchas aristas





Dijkstra

El problema es cuantos vertices DUMMIES se encontrarian?

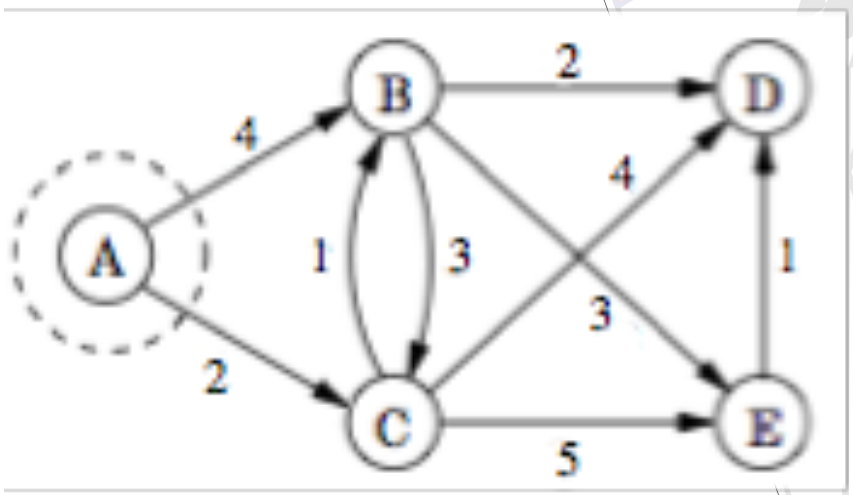


Para solucionar esto se utilizo una “reloj de alarma”, que iba contando los dummies que se pasaban...

Dijkstra

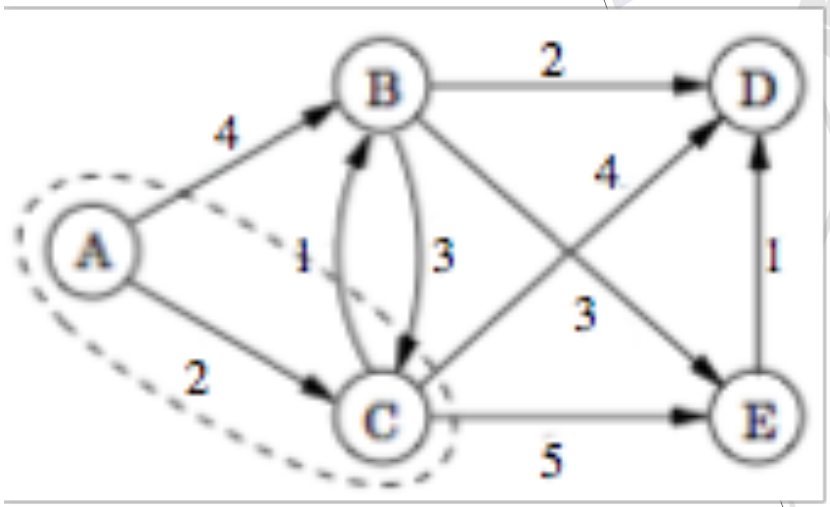
```
procedure dijkstra(G,l,s)
input  : G =(V,E) dirigido o adirigido, l_e : e ∈ E los pesos positivos de las aristas,
         s ∈ V
output: ∀ u ∈ V alcanzable desde s, dist(u) distancia desde s a u
for  u ∈ V do
|    dist(u) = INFINITO
|    prev(u) = nil
end
dist(s) = 0
H = makequeue(V) //Q es una cola prior. distancia la clave
while  H != vacia do
|    u = deletemin(H)
|    for all arista (u,v) ∈ E do
|    |    if  dist(v) > dist(u) + l(u,v) then
|    |    |    dist(v) = dist(u) + l(u,v)
|    |    |    prev(v) = u
|    |    |    decreasekey(H, v)
|    |    end
|    end
end
```

Dijkstra



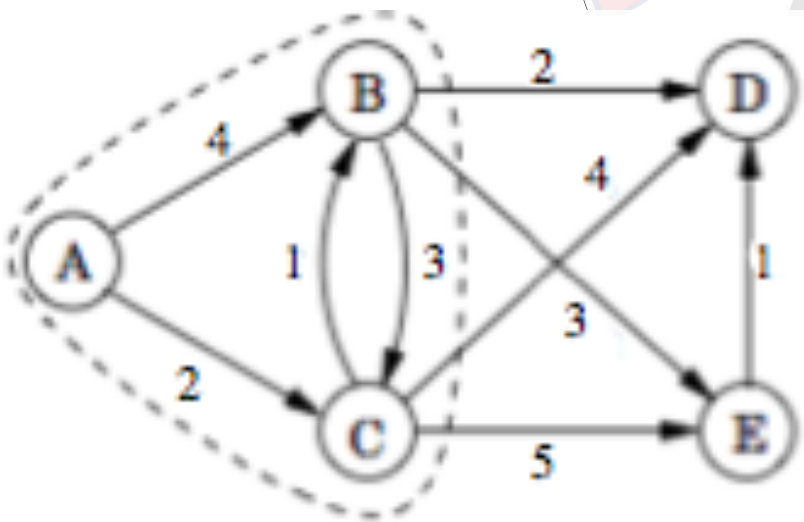
	A	B	C	D	E
dist	0	4	2	IN	IN
prev	/	A	A	/	/

Dijkstra



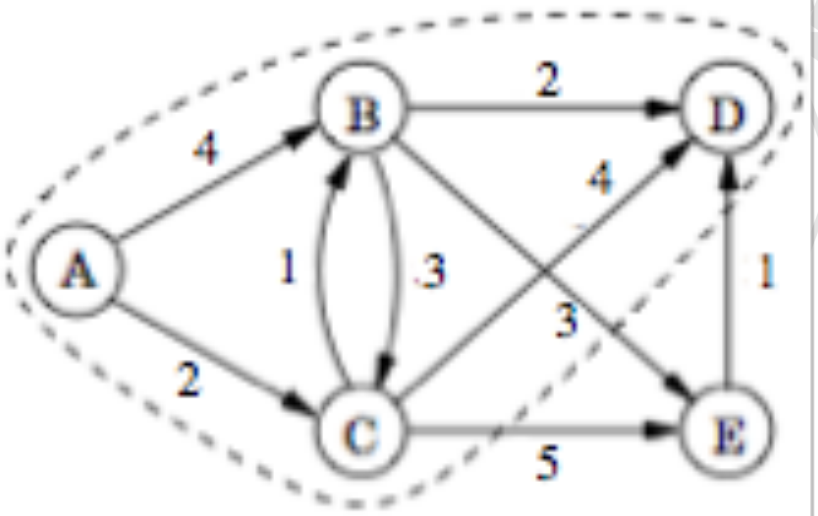
	A	B	C	D	E
dist	0	3	2	6	7
prev	/	C	A	C	C

Dijkstra



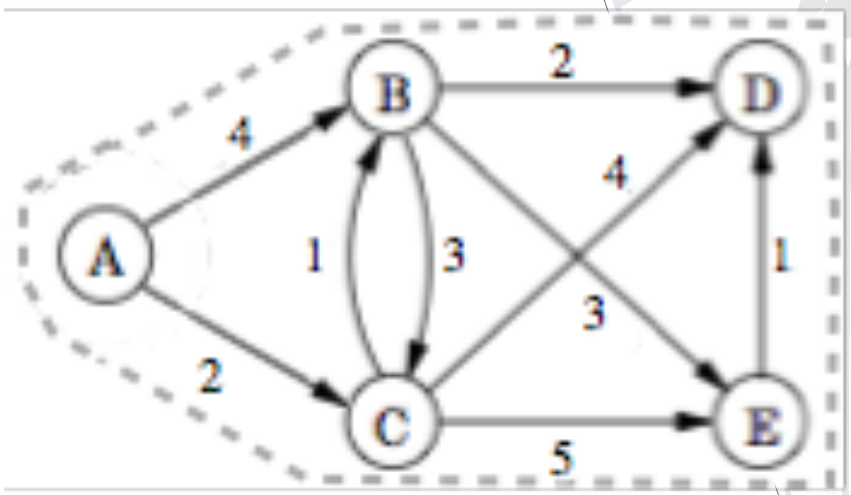
	A	B	C	D	E
dist	0	3	2	5	6
prev	/	C	A	B	B

Dijkstra



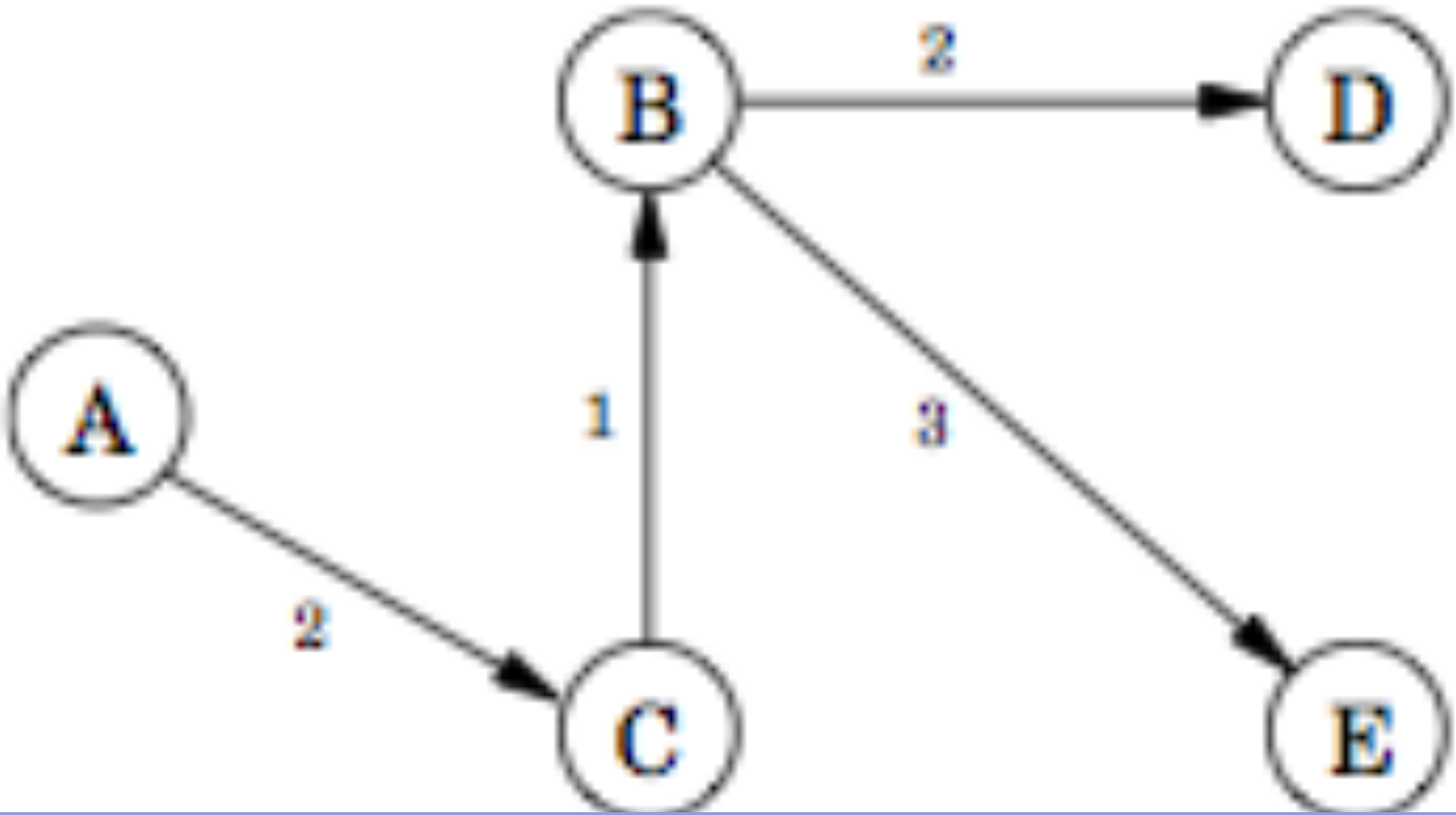
	A	B	C	D	E
dist	0	3	2	5	6
prev	/	C	A	B	B

Dijkstra



	A	B	C	D	E
dist	0	3	2	5	6
prev	/	C	A	B	B

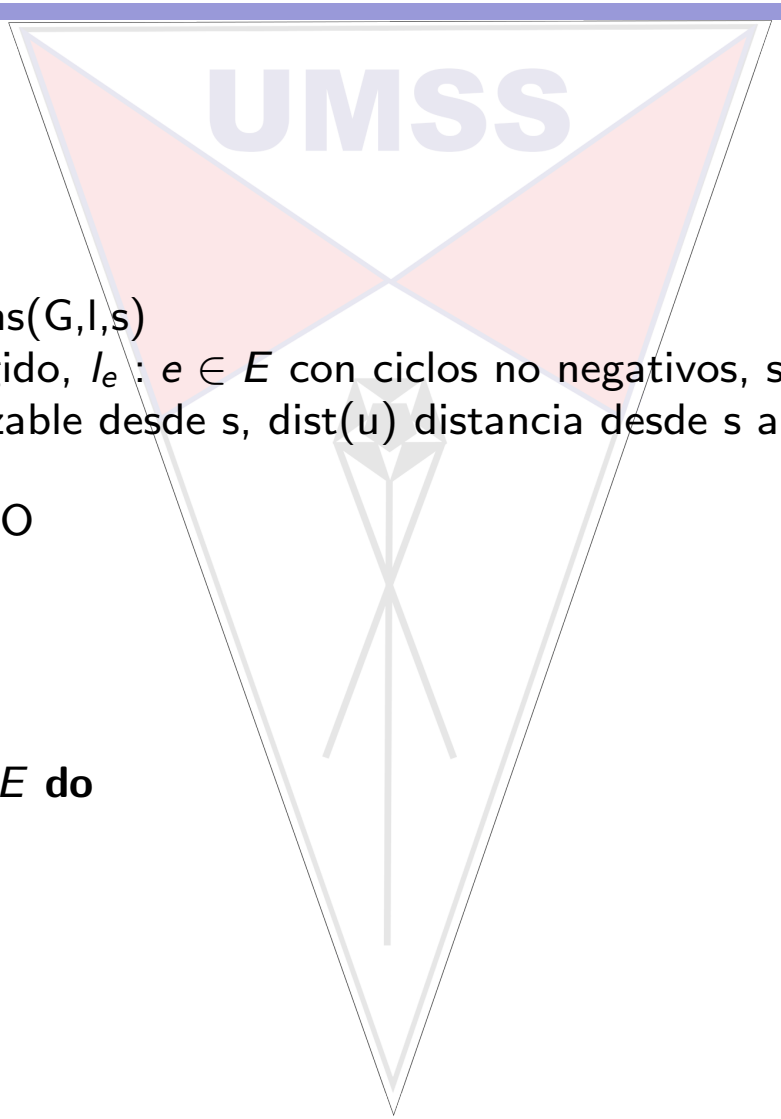
Dijkstra



Bellman Ford

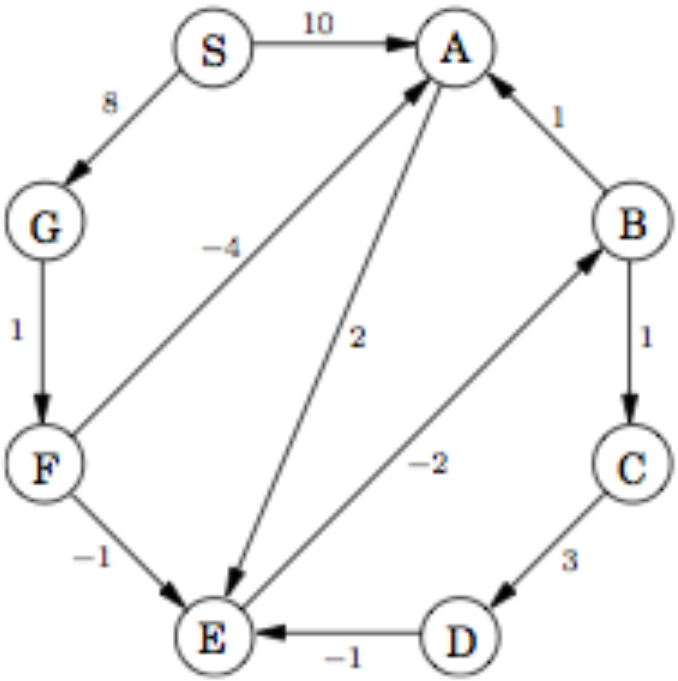
Dijkstra no funciona con aristas negativas....
En este caso, se hace una adaptación de Dijkstra, la misma que cada vez que se analiza una arista se reevaluan las anteriores. Éste es el algoritmo de Bellman Ford

Bellman Ford



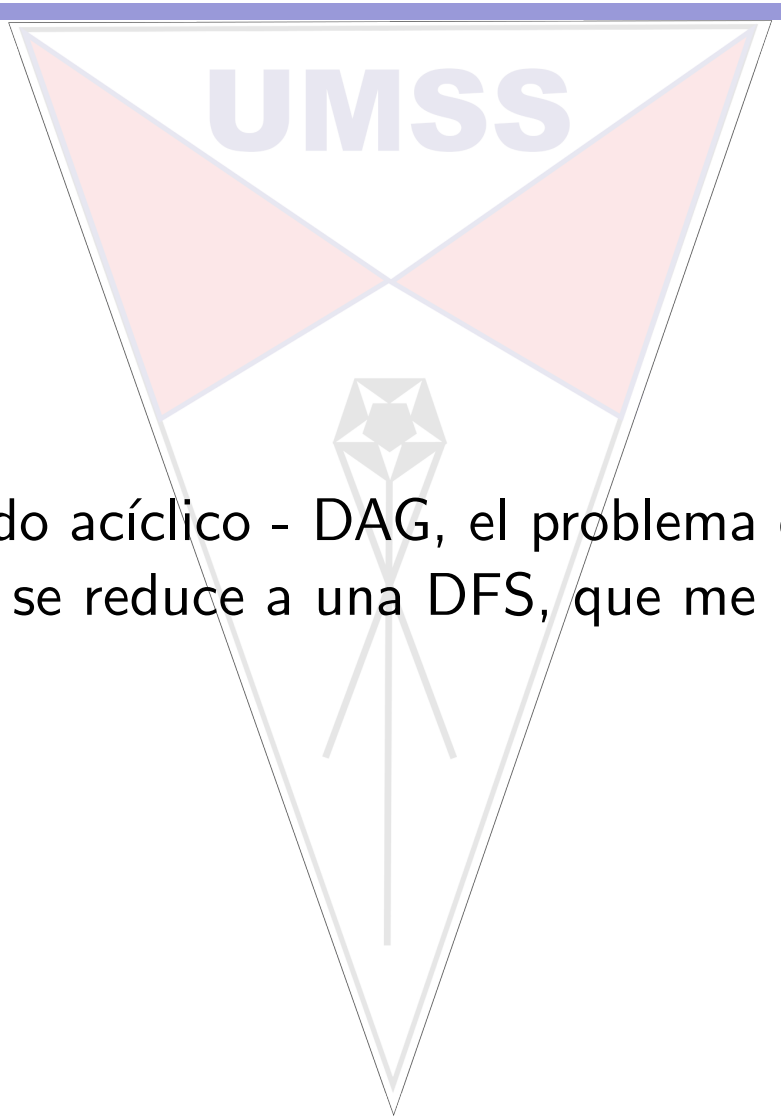
procedure shortest-paths(G,l,s)
input : G =(V,E) dirigido, l_e : $e \in E$ con ciclos no negativos, s $\in$ V
output: $\forall$ u $\in$ V alcanzable desde s, dist(u) distancia desde s a u
for $u \in V$ **do**
  dist(u) = INFINITO
  prev(u) = nil
end
dist(s) = 0
for $|V| - 1$ *veces* **do**
  for $\forall$ $e = (u,v) \in E$ **do**
    update(e)
  end
end

Bellman Ford



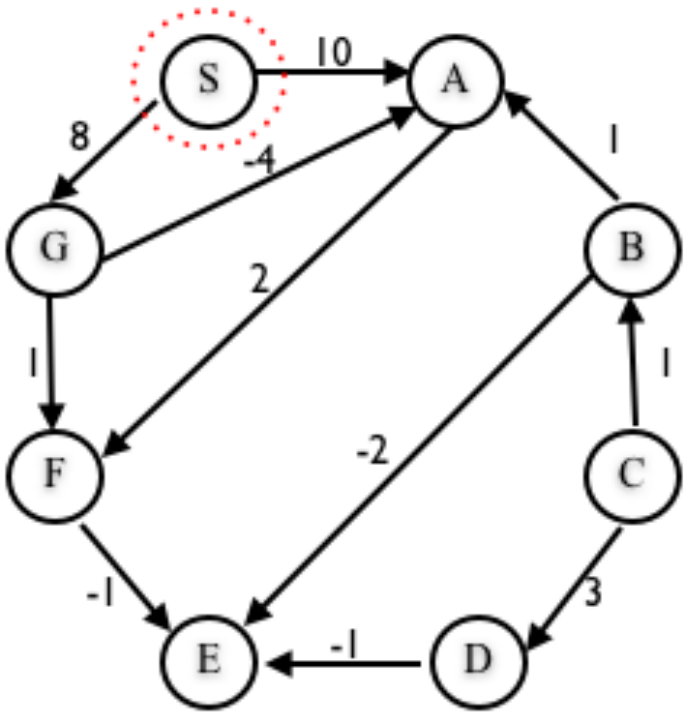
	Iteration							
Node	0	1	2	3	4	5	6	7
S	0	0	0	0	0	0	0	0
A	∞	10	10	5	5	5	5	5
B	∞	∞	∞	10	6	5	5	5
C	∞	∞	∞	∞	11	7	6	6
D	∞	∞	∞	∞	∞	14	10	9
E	∞	∞	12	8	7	7	7	7
F	∞	∞	9	9	9	9	9	9
G	∞	8	8	8	8	8	8	8

DAG



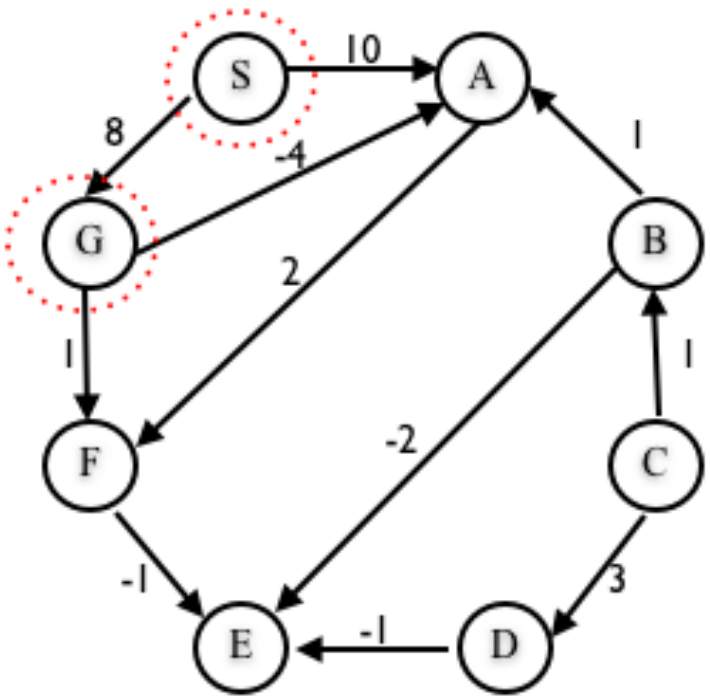
En un grafo dirigido acíclico - DAG, el problema de encontrar el camino mas corto se reduce a una DFS, que me permite encontrar orden topológico.

DAG



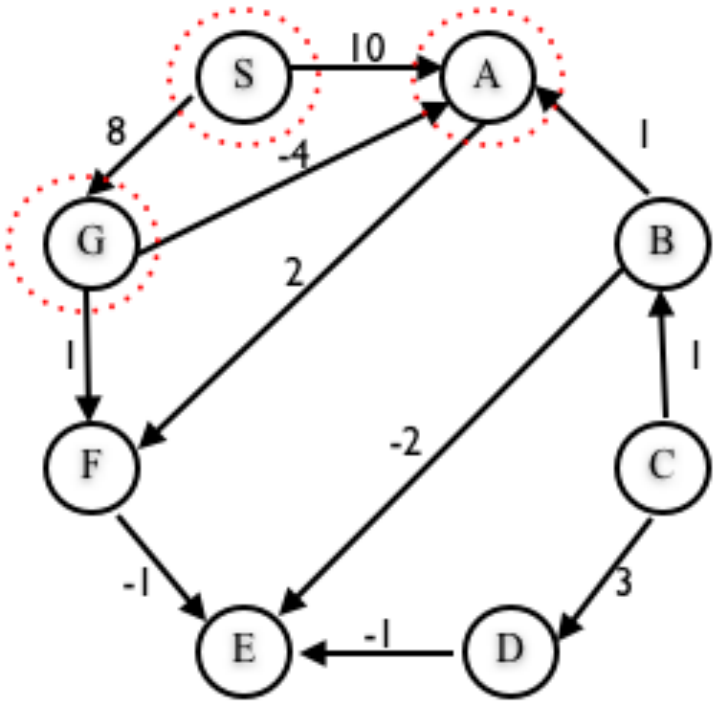
S	A	B	C	D	E	F	G
0	-	-	-	-	-	-	-
/	/	/	/	/	/	/	/

DAG



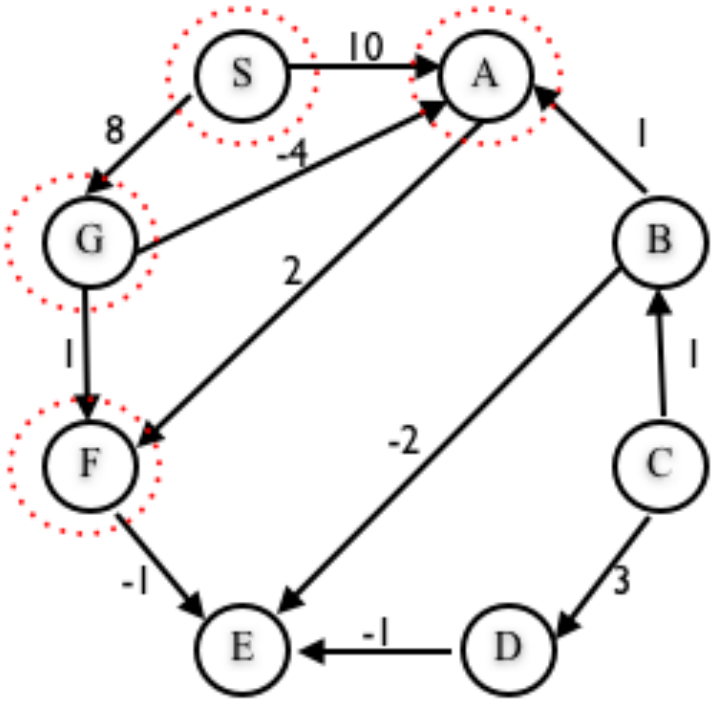
S	A	B	C	D	E	F	G
0	-	-	-	-	-	-	-
/	/	/	/	/	/	/	/
0	10	-	-	-	-	-	8
/	S	/	/	/	/	/	S

DAG



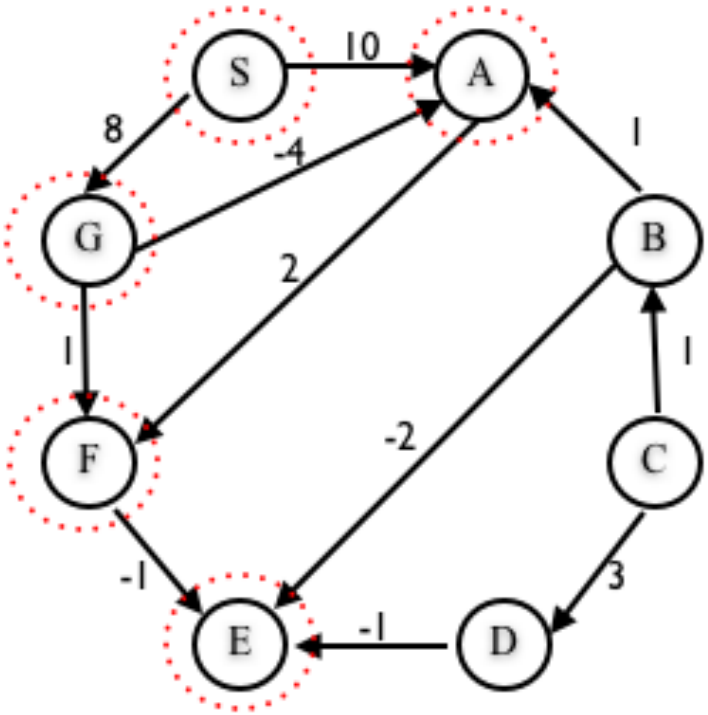
S	A	B	C	D	E	F	G
0	-	-	-	-	-	-	-
/	/	/	/	/	/	/	/
0	10	-	-	-	-	-	8
/	S	/	/	/	/	/	S
0	4	-	-	-	-	9	8
/	G	/	/	/	/	G	S

DAG



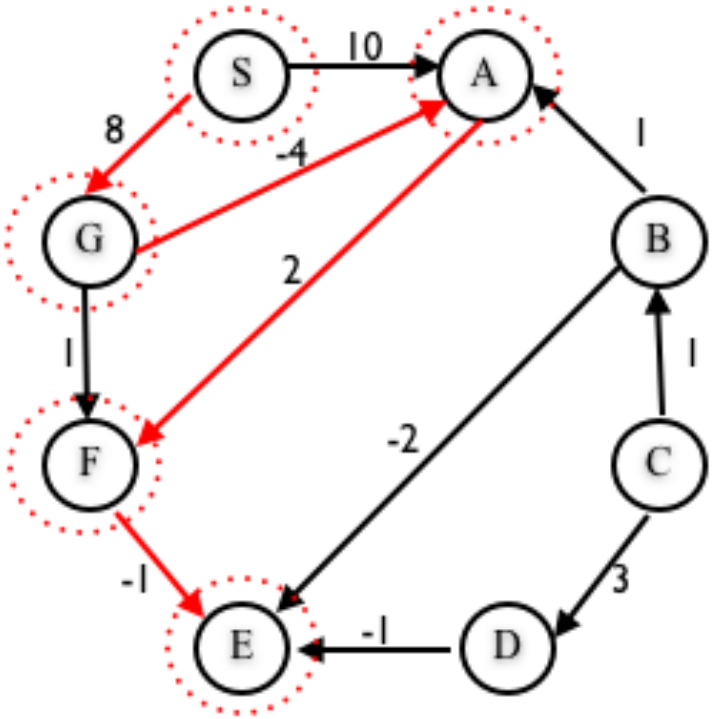
S	A	B	C	D	E	F	G
0	-	-	-	-	-	-	-
/	/	/	/	/	/	/	/
0	10	-	-	-	-	-	8
/	S	/	/	/	/	/	S
0	4	-	-	-	-	9	8
/	G	/	/	/	/	G	S
0	4	-	-	-	-	6	8
/	G	/	/	/	/	A	S

DAG



S	A	B	C	D	E	F	G
0	-	-	-	-	-	-	-
/	/	/	/	/	/	/	/
0	10	-	-	-	-	-	8
/	S	/	/	/	/	/	S
0	4	-	-	-	-	9	8
/	G	/	/	/	/	G	S
0	4	-	-	-	-	6	8
/	G	/	/	/	/	A	S
0	4	-	-	-	5	6	8
/	G	/	/	/	F	A	S

DAG



S	A	B	C	D	E	F	G
0	-	-	-	-	-	-	-
/	/	/	/	/	/	/	/
0	10	-	-	-	-	-	8
/	S	/	/	/	/	/	S
0	4	-	-	-	-	9	8
/	G	/	/	/	/	G	S
0	4	-	-	-	-	6	8
/	G	/	/	/	/	A	S
0	4	-	-	-	5	6	8
/	G	/	/	/	F	A	S